

1. Bagaimana Frontend & Backend Berhubungan?

Komunikasi antara frontend (React) dan backend (Express/Prisma) dilakukan secara dinamis menggunakan REST API melalui library Axios. Berikut adalah alur hubungannya:

1. Autentikasi (JWT Token):

- Saat user melakukan login, backend mengirimkan *JSON Web Token (JWT)*. Token ini disimpan di browser oleh frontend pada `localStorage.getItem('token')`.
- Setiap kali frontend memanggil API transaksi, token ini wajib disisipkan di dalam header request sebagai authorization header: `Authorization: Bearer <token>`.

2. Pengambilan & Manipulasi Data:

- Frontend mengirimkan request (GET/POST/DELETE) ke URL backend (seperti `http://localhost:5000/api/...`).
- Backend memproses data tersebut ke database SQLite (via Prisma) dan mengembalikan respons dalam format JSON.
- Frontend menerima JSON tersebut, menyimpannya ke dalam *state* React (`useState`), lalu melakukan render ulang (*re-render*) untuk menampilkan data terbaru ke layar user.

2. Detail Frontend (Halaman Showcase & Portofolio)

Halaman-halaman ini bersifat client-side presentation (tidak menembak API backend karena fokus pada performa render cepat dan visual premium). Data disuplai dari berkas data statis `divisions.js`.

1. Divisi Free Fire (FF)

- Berkas: `ff.jsx`

Baris 62: 1. HERO SECTION & STATISTIK RINGKASAN

Fungsi: Menampilkan banner atas, tombol kembali, dan metrik performa utama (Win Rate, Event, Peringkat).

Baris 126: 2. ROSTER UTAMA (MAIN ROSTER)

Fungsi: Menampilkan daftar pemain inti beserta role, jumlah kills, dan rating individual performa mereka.

Baris 162: 3. PEMAIN CADANGAN & AKADEMI (SUB ROSTER)

Fungsi: Menampilkan daftar pemain pengganti dan pemain akademi beserta role masing-masing.

Baris 187: 4. PAPAN PERINGKAT KLASEMEN (RANKINGS)

Fungsi: Menampilkan tabel klasemen turnamen yang berisi rank, nama tim rival, total poin, dan rekor menang-kalah.

Baris 232: 5. KARTU INFORMASI TIM (TEAM PROFILE)

Fungsi: Menampilkan detail profil tim Hilling Strike, seperti tahun didirikan, total kemenangan, dan kekalahan match.

Baris 263: 6. STAFF & PELATIH (COACH SECTION)

Fungsi: Menampilkan profil pelatih kepala dan asisten pelatih, termasuk nama, jabatan, dan spesialisasi mereka.

Baris 291: 7. JADWAL PERTANDINGAN (SCHEDULE MATCH)

Fungsi: Menampilkan jadwal laga tanding mendatang lengkap dengan tanggal, waktu, turnamen, dan tim lawan (format VS).

2. Divisi Honor of Kings (HOK)

- Berkas: hok.jsx

Baris 39: 1. HERO SECTION & STATISTIK RINGKASAN

Fungsi: Menampilkan banner game Honor of Kings dan ringkasan metrik seperti total roster, jumlah turnamen, dan pencapaian juara.

Baris 80: 2. FORMASI PEMAIN UTAMA (MAIN ROSTER)

Fungsi: Menampilkan kartu formasi pemain inti berdasarkan layout lane (Clash, Jungle, Mid, Farm, Roam) beserta statistik Kills & Rating.

Baris 128: 3. STAF PELATIH & FILOSOFI (COACH SECTION)

Fungsi: Menampilkan profil pelatih kepala dan asisten pelatih, termasuk nama, spesialisasi, dan pengalaman mereka.

Baris 151: 4. STRATEGI UTAMA PERMAINAN (TACTICAL STRATEGY)

Fungsi: Menjelaskan fokus taktik tim dalam game, seperti kontrol objektif (turtle/lord) dan penentuan draft hero.

Baris 173: 5. PRESTASI & PENCAPAIAN TIM (TEAM ACHIEVEMENTS)

Fungsi: Menampilkan jumlah kemenangan match dan persentase win rate tim HOK Hilling Strike secara visual.

Baris 191: 6. JADWAL LAGA TANDING (UPCOMING MATCHES)

Fungsi: Menampilkan daftar jadwal tanding tim HOK di turnamen mendatang beserta detail tanggal, waktu, dan nama tim lawan.

3. Halaman Tentang Kami (About)

- Berkas: About.jsx

Baris 19: 1. HERO SECTION & INTRODUKSI UTAMA

Fungsi: Menampilkan tajuk selamat datang, deskripsi profil Hilling Strike, dan logo tim dengan animasi denyut (pulse).

Baris 55: 2. STATISTIK PENCAPAIAN ORGANISASI

Fungsi: Menampilkan metrik pencapaian organisasi seperti total divisi game, jumlah pro player, total piala juara, dan jumlah estimasi fans.

Baris 78: 3. VISI & MISI ORGANISASI (FILOSOFI KAMI)

Fungsi: Menyajikan filosofi arah masa depan tim (Visi) dan langkah-langkah konkret pembinaan talenta & prestasi yang dijalankan (Misi).

3. Detail Backend (Endpoints Transaksi)

Backend menangani logika bisnis transaksi, validasi stok, pencatatan pembayaran, dan riwayat pesanan.

A. Proses Checkout (Pembelian)

- Lokasi Backend: checkout.js (*Baris 10 - 95*)
- Lokasi Pemicu di Frontend: Store.jsx (*Baris 124 - 141*) di dalam fungsi processCheckout.
- Alur Hubungan:
 1. Ketika user mengklik "*SIMULASIKAN PEMBAYARAN*" di modal Midtrans, frontend memanggil fungsi processCheckout(paymentMethod).
 2. Frontend menembak endpoint POST `http://localhost:5000/api/checkout` dengan membawa data keranjang (cartItems) dan metode pembayaran di dalam *request body*, serta JWT token di *headers*.
 3. Backend menerima request, memverifikasi token user, lalu memulai transaksi database (prisma.\$transaction) untuk:
 - Cek dan kurangi stok barang di database.
 - Membuat baris baru di tabel pesanan, detail_pesanan, dan tabel pembayaran (status langsung PAID).
 - Menghapus isi keranjang di database (keranjang).
 4. Backend merespons dengan status sukses. Frontend menerima respons itu dan mengosongkan keranjang belanja lokal, dan mengalihkan halaman ke /history (Riwayat Transaksi).

B. Riwayat Transaksi (History)

- Lokasi Backend: transactions.js (*Baris 9 - 30*)
- Lokasi Konsumsi di Frontend: History.jsx (*Baris 11 - 29*) di dalam fungsi fetchHistory.
- Alur Hubungan:
 1. Begitu halaman Riwayat dimuat, React menjalankan useEffect yang memanggil fungsi fetchHistory().
 2. Frontend menembak GET `http://localhost:5000/api/transactions/history` dengan authorization token.
 3. Backend mencocokkan ID user dari token JWT, lalu mengambil data pesanan, detail produk, dan status bayar dari database SQLite, mengurutkannya dari yang terbaru, dan mengirimkannya kembali ke frontend.
 4. Frontend menyimpan data tersebut ke state history dan menampilkannya dalam bentuk kartu riwayat transaksi yang rapi.

C. Pembatalan Transaksi (Cancel & Pengembalian Stok)

- Lokasi Backend: transactions.js
- Lokasi Pemicu di Frontend: History.jsx di dalam fungsi handleCancelOrder.
- Alur Hubungan:
 1. User mengklik tombol "BATALKAN PESANAN" pada salah satu pesanan di halaman riwayat.(Baris 197 - 205)
 2. Frontend meminta konfirmasi (`window.confirm`), (Baris 45 – 47)

lalu mengirim request POST ke backend dengan menyertakan parameter ID pesanan: POST <http://localhost:5000/api/transactions/cancel/:idPesanan>. (baris 50-52)
 3. Backend menerima request dan menjalankan database transaction (Baris 37 - 38):
 - Mengubah status pesanan dan pembayaran menjadi CANCELLED. (Baris 65 - 74)
 - Mengambil item produk di dalam pesanan tersebut, lalu menambahkan kembali kuantitas produk tersebut ke stok utama agar jumlah stok produk di database kembali utuh. (Baris 76 - 92)

4. Backend mengirim pesan sukses. Frontend memunculkan alert sukses, lalu memanggil kembali `fetchHistory()` untuk memperbarui tampilan riwayat pesanan menjadi "BATAL". 11:22 PM (Baris 53 - 54)

Pertanyaan yang mungkin terjadi

"Kenapa Anda menggunakan `$transaction` pada proses Checkout dan Pembatalan Pesanan?"

jawab:

"Kami menggunakan `$transaction` untuk memastikan bahwa semua proses berjalan sebagai satu kesatuan yang utuh. Jika salah satu proses gagal (misalnya stok produk ternyata tidak mencukupi atau database error di tengah jalan), maka seluruh proses lainnya akan dibatalkan otomatis (rollback). Ini mencegah timbulnya data 'sampah', seperti pesanan yang tercatat padahal pembayaran gagal atau stok yang berkurang tanpa adanya pesanan."

- Referensi Lokasi: `checkout.js` Baris 23 dan `transactions.js` Baris 63.

"Bagaimana cara sistem Anda memastikan stok produk tidak menjadi minus saat user melakukan checkout?"

jawab:

"Sebelum memotong stok dan membuat pesanan, backend melakukan pemeriksaan stok terlebih dahulu di berkas `checkout.js` pada Baris 24 - 34. Sistem mencari produk terkait di database, lalu membandingkan stok yang tersedia dengan jumlah yang diminta oleh user. Jika jumlah yang dibeli melebihi stok yang tersedia, backend akan melempar error Stok tidak mencukupi dan membatalkan seluruh transaksi."

- Referensi Lokasi: `checkout.js` Baris 24-34.

"Bagaimana backend memastikan bahwa proses pembatalan pesanan aman (tidak bisa membatalkan pesanan milik orang lain)?"

jawab:

"Kami menggunakan middleware `verifyToken` di Baris 37 berkas `transactions.js` untuk mengekstrak ID User dari token JWT yang dikirim oleh frontend. Kemudian, saat mencari pesanan yang ingin dibatalkan pada Baris 43 - 47, kami menggunakan kriteria pencarian kombinasi `id_pesanan` dan `id_user`. Jadi, jika user mencoba membatalkan ID pesanan yang bukan miliknya, database tidak akan menemukannya dan request akan langsung ditolak."

- Referensi Lokasi: `transactions.js` Baris 37 & 43-47.

Pertanyaan Seputar Frontend (About, FF, HOK)

"Dari mana data roster pemain, coach, dan peringkat di halaman FF dan HOK diambil? Apakah dinamis dari API database?"

jawab:

"Untuk halaman portofolio divisi game (FF dan HOK) serta halaman About, datanya bersifat statis dan diambil dari berkas divisions.js. Hal ini sengaja dilakukan agar halaman showcase tim dapat dimuat dengan sangat cepat (high performance) tanpa perlu melakukan request berulang ke server backend, karena data profil tim dan roster relatif jarang berubah."

- Referensi Lokasi: Berkas data utama di divisions.js.

"Bagaimana cara Anda menampilkan foto pemain secara dinamis padahal datanya di-import secara lokal di bagian atas berkas?"

jawab:

"Kami menggunakan teknik mapping objek di React. Kami meng-import file gambar lokal di bagian atas berkas, lalu membuat objek pemetaan bernama playerImageMap. Saat melakukan perulangan data pemain menggunakan fungsi .map(), sistem akan mencocokkan nama pemain dari file data divisions.js dengan kunci di dalam objek playerImageMap untuk mengambil asset gambar yang sesuai."

- Referensi Lokasi: Objek pemetaan playerImageMap di berkas ff.jsx Baris 19-31 dan hok.jsx Baris 16-22.